



Disciplina: **FGA0314 – Testes de Software**

Dublês de Teste

Elaine Venson

elainevenson@unb.br

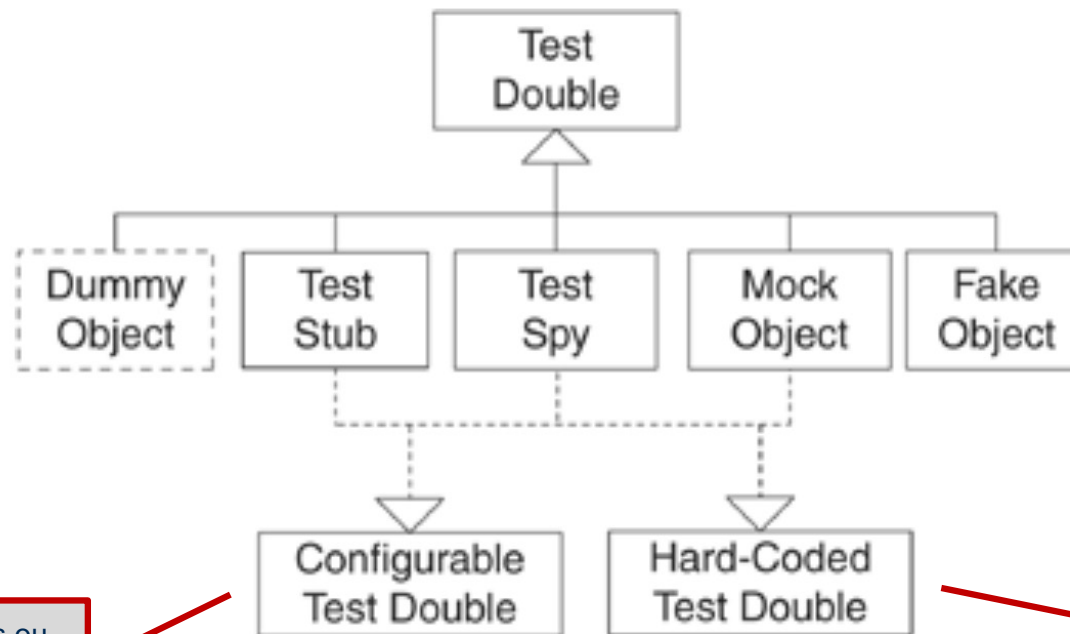


Dublês de Teste

- Termo cunhado por Gerard Meszaros no livro *xUnit Test Patterns*
- Um dublê de teste é qualquer objeto ou componente que é instalado no lugar do componente real com o propósito de executar um teste
- São utilizados para isolar a unidade de teste de outros componentes dos quais ele depende



Tipos de Dublês



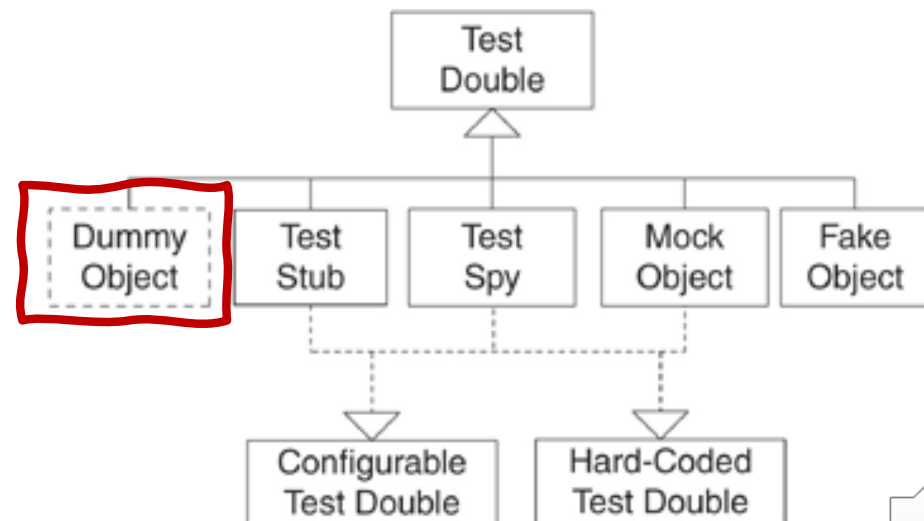
Valores retornados ou esperados são definidos em tempo de execução pelo teste

Valores retornados ou esperados são definidos na implementação do dublê



Dummy Object

- Um objeto que é passado para o SST como um parâmetro (ou atributo de um parâmetro), mas nunca é utilizado.
- Utilizado quando o código espera um objeto real, não nulo, mas não se espera que ele faça algo.



* SST: software sendo testado



Dummy Object - Exemplo

```
Book javaBook = new Book("Java 101", "123456");  
Member dummyMember = new DummyMember();  
javaBook.issueTo(dummyMember);  
assertEquals(javaBook.numberOfTimesIssued(),1);
```

Teste



Dummy Object - Exemplo

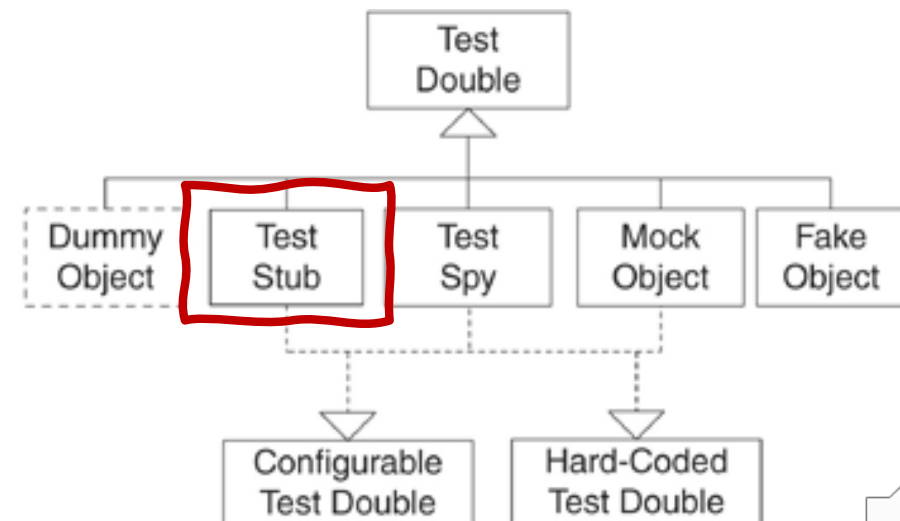
```
public void testInvoice_addLineItem_DO() {  
    final int QUANTITY = 1;  
    Product product = new Product("Dummy Product Name",  
                                   getUniqueNumber());  
  
    Invoice inv = new Invoice( new DummyCustomer() );  
    LineItem expItem = new LineItem(inv, product, QUANTITY);  
    // Exercise  
    inv.addItemQuantity(product, QUANTITY);  
    // Verify  
    List lineItems = inv.getLineItems();  
    assertEquals("number of items", lineItems.size(), 1);  
    LineItem actual = (LineItem)lineItems.get(0);  
    assertLineItemsEqual("", expItem, actual);  
}
```

Teste



Test Stubs

- Um objeto que atua como um ponto de controle para entregar dados de entrada indiretos para o SST quando os métodos de *Test Stub* são chamados.
- Permite testar caminhos de execução do código do SST que seria impossível sem o stub.
- Tipos:
 - *Responder*: retorna valores válidos e inválidos
 - *Saboteur*: retorna exceções



* SST: software sendo testado



Test Stubs - Exemplo

```
public interface Dispenser {  
    void dispense(BigDecimal amount) throws DispenserFailed;  
}
```

```
public class AlwaysFailingDispenserStub implements Dispenser{  
    public void dispense(BigDecimal amount) throws  
DispenserFailed{  
        throw new DispenserFailed (ErrorType.HARDWARE,"not  
responding");  
    }  
}
```

Stub Saboteur

```
class ATMTTest...  
    @Test  
    public void transaction_is_rolledback_when_hardware_fails()  
{  
        Account myAccount = new Account("John", 2000.00);  
        TransactionManager txMgr =  
TransactionManager.forAccount(myAccount);  
        txMgr.registerMoneyDispenser(new  
AlwaysFailingDispenserStub());  
        WithdrawalResponse response = txMgr.withdraw(500.00);  
        assertEquals(false, response.wasSuccess());  
        assertEquals(2000.00, myAccount.remainingAmount());  
    }  
}
```

Teste



Test Stubs – Exemplo

```
public void testDisplayCurrentTime_exception()
    throws Exception {
    // Fixture setup
    // Define and instantiate Test Stub
    TimeProvider testStub = new TimeProvider()
    { // Anonymous inner Test Stub
        public Calendar getTime() throws TimeProviderEx {
            throw new TimeProviderEx("Sample");
        }
    };
    // Instantiate SUT
    TimeDisplay sut = new TimeDisplay();
    sut.setTimeProvider(testStub);
    // Exercise SUT
    String result = sut.getCurrentTimeAsHtmlFragment();
    // Verify direct output
    String expectedTimeString =
        "<span class=\"error\">Invalid Time</span>";
    assertEquals("Exception", expectedTimeString, result);
}
```

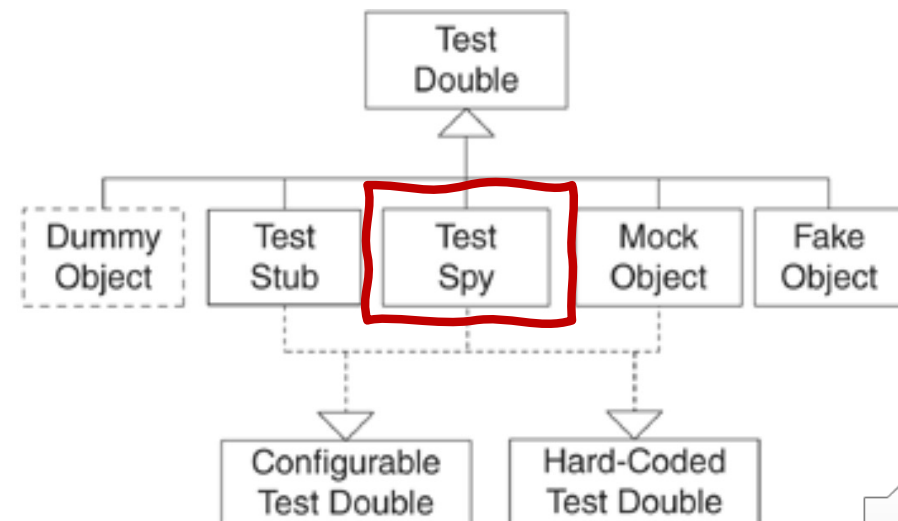
*Stub Saboteur
(anonymous inner class)*

Teste



Test Spy

- Um objeto que atua como um ponto de observação das saídas de dados indiretas do SST.
- É um stub com o recurso adicional de registrar de forma transparente todas as chamadas feitas pelo SST.
- Compara as chamadas realizadas com as chamadas esperadas.



* SST: software sendo testado



Test Spy - Exemplo

Código a ser testado

```
class ResourceAdapter{
    void print(String userId, String document, Object settings)
    {
        if(securityService.canAccess("lanPrinter1", userId)) {
            printer.print(document, settings);
        }
    }
}
```

Test Spy para verificar se o método *print* é chamado quando o usuário tem permissão

```
class SpyPrinter implements Printer{
    private int noOfTimescalled = 0;
    @Override
    public void print(Object document, Object settings) {
        noOfTimescalled++;
    }
    public int getInvocationCount() {
        return noOfTimescalled;
    }
}
```

Implementação fake de Security Service para retornar true

```
class FakeSecurityService implements SecurityService{
    public boolean canAccess(String printerName, String userId){
        return true;
    }
}
```

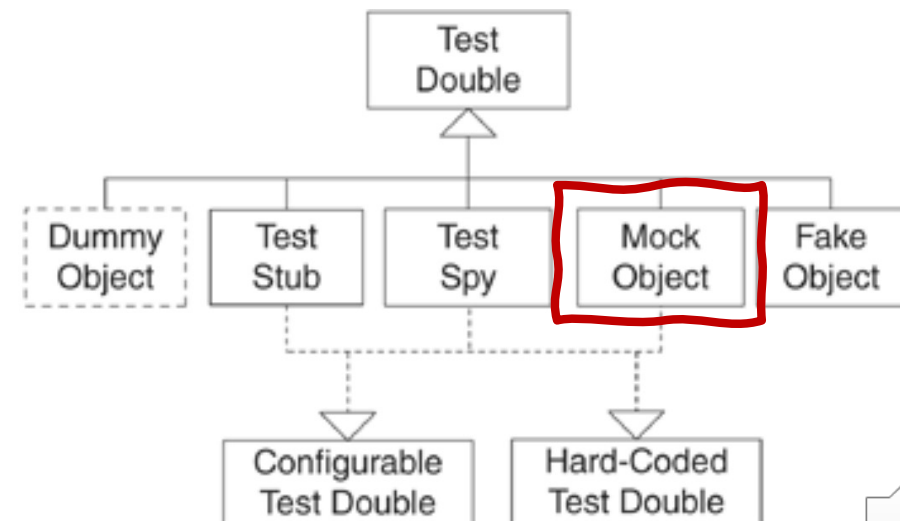
Teste

```
@Test public void verify() throws Exception {
    SpyPrinter spyPrinter = new SpyPrinter();
    adapter = new ResourceAdapter(new FakeSecurityService(),
    spyPrinter);
    adapter.print("john", "helloWorld.txt", "all pages");
    assertEquals(1, spyPrinter.getInvocationCount());
}
```



Mock Objects

- Um objeto que também atua como um ponto de observação dos dados de saída indiretos do SST. Como um stub pode também responder a chamadas de métodos. Diferente do Test Spy, o Mock Object compara as chamadas recebidas com as chamadas esperadas usando *assertions* e falha o teste em nome do método de teste.
- Permite reusar a lógica para verificar as saídas indiretas de todos os testes que usam o mesmo Mock Object.
- Tipos:
 - *Strict Mock Object*: falha se as chamadas são recebidas em uma ordem diferente da esperada
 - *Lenient Mock Object*: tolera chamadas fora de ordem



* SST: software sendo testado



Mock Objects – Exemplo

hand-build

```
public void testRemoveFlight_Mock() throws Exception {  
    // Fixture setup  
    FlightDto expectedFlightDto = createAnonRegFlight();  
    // Mock configuration  
    ConfigurableMockAuditLog mockLog =  
        new ConfigurableMockAuditLog();  
    mockLog.setExpectedLogMessage(  
        helper.getTodaysDateWithoutTime(),  
        Helper.TEST_USER_NAME,  
        Helper.REMOVE_FLIGHT_ACTION_CODE,  
        expectedFlightDto.getFlightNumber());  
    mockLog.setExpectedNumberCalls(1);  
    // Mock installation  
    FlightManagementFacade facade =  
        new FlightManagementFacadeImpl();  
    facade.setAuditLog(mockLog);  
    // Exercise  
    facade.removeFlight(expectedFlightDto.getFlightNumber());  
    // Verify  
    assertFalse("flight still exists after being removed",  
        facade.flightExists( expectedFlightDto.  
            getFlightNumber()));  
    mockLog.verify();  
}
```

configura um Mock Object com os
parâmetros esperados da chamada
para *logMessage()*

```
public void logMessage( Date actualDate,  
    String actualUser,  
    String actualActionCode,  
    Object actualDetail) {  
    actualNumberCalls++;  
  
    Assert.assertEquals("date", expectedDate, actualDate);  
    Assert.assertEquals("user", expectedUser, actualUser);  
    Assert.assertEquals("action code",  
        expectedActionCode,  
        actualActionCode);  
    Assert.assertEquals("detail", expectedDetail, actualDetail);  
}
```

Implementação
manual do mock

verifica se a chamada para
logMessage() foi realizada



Mock Objects – Exemplo

usando Mockito

```
public class ATMTTest {
    @Mock Dispenser failingDispenser;

    @Before public void setUp() throws Exception {
        MockitoAnnotations.initMocks(this);
    }

    @Test public void
transaction_is_rolledback_when_hardware_fails() throws
DispenserFailed {
        Account myAccount = new Account(2000.00, "John");
        TransactionManager txMgr =
TransactionManager.forAccount(myAccount);
        txMgr.registerMoneyDispenser(failingDispenser);

        doThrow(new
DispenserFailed()).when(failingDispenser).dispense(isA(BigDecimal.class));

        txMgr.withdraw(500);
        assertEquals(2000.00, myAccount.getRemainingBalance());

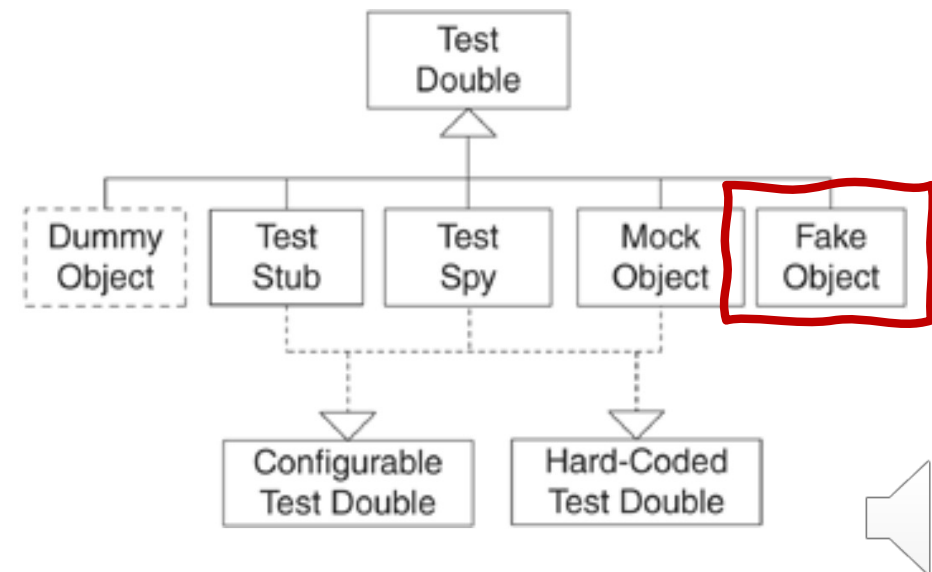
        verify(failingDispenser, new
Times(1)).dispense(isA(BigDecimal.class));
    }
}
```

Lança exceção
sempre que o mock
object é chamado
com qualquer valor
BigDecimal



Fake Objects

- Usado para substituir a funcionalidade de um componente real, por razões diferentes de verificar entradas e saídas indiretas.
- Geralmente um *Fake Object* implementa uma funcionalidade ou subconjunto de uma funcionalidade de um componente real de uma forma mais simples.
- Usado quando o componente real ainda não foi construído, é muito lento ou não está disponível no ambiente de teste.



* SST: software sendo testado



Fake Objects: Exemplo

Fake Database:

encapsulamento de todo o acesso ao banco de dados em uma camada de persistência e posterior substituição da camada de persistência por um componente fake que usa tabelas hash na memória em vez de um banco de dados real para tornar a execução dos testes mais rápida.

Implementação fake da camada de persistência

```
public class InMemoryDatabase implements FlightDao{
    private List airports = new Vector();
    public Airport createAirport(String airportCode,
                                String name, String nearbyCity)
        throws DataException, InvalidArgumentException {
        assertParamtersAreValid( airportCode, name, nearbyCity);
        assertAirportDoesntExist( airportCode);
        Airport result = new Airport(getNextAirportId(),
                                    airportCode, name, createCity(nearbyCity));
        airports.add(result);
        return result;
    }
    public Airport getAirportByPrimarykey(BigDecimal airportId)
        throws DataException, InvalidArgumentException {
        assertAirportNotNull(airportId);
        Airport result = null;
        Iterator i = airports.iterator();
        while (i.hasNext()) {
            Airport airport = (Airport) i.next();
            if (airport.getId().equals(airportId)) {
                return airport;
            }
        }
        throw new DataException("Airport not found:"+airportId);
    }
}
```



Atividade:

Identificando e Classificando Mock Objects no Projeto Open Source

Objetivo

Reconhecer, no projeto open source da equipe:

- onde existem dependências externas;
- quais tipos de dublês de teste (test doubles) estão sendo usados;
- como os mocks ajudam no isolamento dos testes.

Cada equipe deverá analisar os testes existentes do projeto open source escolhido e identificar exemplos de:

- Mock
- Stub
- Fake
- Spy
- Dummy

(se existirem)

Referências

- Acharya, Sujoy. Mastering Unit Testing Using Mockito and JUnit, Packt Publishing Ltd., 2014. (Capítulo 3)
- Meszaros, Gerard. xUnit Test Patterns: Refactoring Test Code, 2007. (Capítulo 11)

